Nye WordPress-sites

guide, metode og sammenligning

Skrevet: 8/11 2026 · **Gælder:** de næste sites vi starter, ikke steamgears.eu · **Kilder:** §13

Denne guide svarer på to ting: **hvad er den bedste og mest moderne måde at bygge et WordPress-site på i dag**, og **hvordan står den metode mod den vi selv bruger**. Den er skrevet, så et nyt site kan startes på under to uger uden at gentage de fejl steamgears har betalt for.

Den erstatter ikke deploy-playbook-2026-11-08.md, der handler om at publicere **en side** på et site der allerede kører. Denne handler om at **vælge og bygge sitet**.

---

0. Kort opsummering

Den moderne metode (Git- og staging-metoden) har fem dele, og de hænger sammen:

1. **Prototype i WordPress Playground** med en Blueprint-JSON i repoet — et helt site på sekunder, uden server, perfekt til demo og kundegodkendelse.
2. **Byg lokalt** med `wp-env`, DDEV eller WordPress Studio, med WP-CLI og PHP i samme version som drift.
3. **Block theme med theme.json** — ikke page builder. Designet er kode, ikke 900 indstillinger i en database.
4. **Git → staging → live.** Hostens eget Git-deploy eller en CI-pipeline over SSH; staging er en klon, live er beskyttet, og rollback er en tidligere release.
5. **Managed drift:** backup med testet restore, WAF/CDN, object cache, automatiske opdateringer der først rammer staging, og WP-CLI/SSH til fejlfinding.

Vores metode er Simply + wp-admin/MCP + et separat dokumentationsrepo. Den har **ingen staging, ingen Git-deploy af site-koden og ingen lokal reproduktion**, men til gengæld en måle- og værnkultur der er bedre end de fleste: hver side måles, hver commit bærer procenter, og hver fælde bliver et værn.

Konklusion i tre sætninger: til **nye sites** er Git- og staging-metoden det rigtige valg — den koster mere pr. måned og kræver SSH/Git-kompetence, men den fjerner præcis de tre svagheder der har kostet os dyrest (ingen staging, ingen rollback af kode, fejlfinding gennem en cache). Til **steamgears** skal vi ikke migrere for migrationens skyld; siden kører, er målt, og en flytning ville koste mere end den giver lige nu. Uanset hvad vi vælger, tager vi **værnene og dokumentationen** med — de er metoden, ikke hosten.

Spørgsmål	Vores metode	Git- og staging-metoden
Kan en ændring fortrydes på sekunder?	Nej, ikke kode	Ja, forrige release
Kan en ændring testes før live?	Nej	Ja, på staging
Er site-koden versionsstyret?	Kun plugin'et	Ja, hele temaet og egen kode
Kan sitet genskabes lokalt?	Nej	Ja, med kommando
Koster det mere pr. måned?	Nej	Ja, typisk 2–10× shared
Kræver det mere af den der driver det?	Nej	Ja: Git, SSH, deploy

---

1. Den moderne metode — de seks lag

Lag	Værktøj i 2026	Hvorfor	Bevis
0. Beslut	indholdsmodel, slugs, redirects	slug-bytte efter live kræver 301	egen erfaring, deploy-playbook §3.0
1. Prototype	Playground + Blueprint-JSON	demo på sekunder, samme udgangspunkt hver gang	Playground Handbook, Blueprints Docs
2. Lokal	wp-env, DDEV, WordPress Studio	teste PHP/plugins før de rammer nogen	Theme Handbook: Tools and Setup
3. Tema	block theme + theme.json + patterns	design som kode, ingen builder-lås	Theme Handbook, 7.1 dev notes
4. Kode	Git-repo (tema, egen plugin, mu-plugins)	historik, review, rollback	Bedrock-dokumentation
5. Deploy	hostens Git-deploy eller CI over SSH	staging → live med et klik eller en merge	WP Pusher-hostmatrix 20/10 2026
6. Drift	backup + restore-øvelse, WAF, object cache, WP-CLI	fejl skal findes i logs, ikke i cachen	Kinsta/Raidboxes-dokumentation

1.1 Prototype: Playground er blevet en rigtig arbejdsflade

WordPress Playground kører WordPress i browseren eller i Node via PHP-WASM. I 2026 fik det en ny grænseflade til flere sites, en samlet **Playground Handbook** og en **Site Manager API**, der kan oprette, liste, ændre og slette Playground-sites fra kode — altså scriptede demoer og testmatricer uden en server. En Blueprint er en JSON-fil, der beskriver hele opsætningen: versioner, tema, plugins, indhold, indstillinger og login.

```
{
  "$schema": "https://playground.wordpress.net/blueprint-schema.json",
  "preferredVersions": { "php": "8.3", "wp": "latest" },
  "landingPage": "/wp-admin/",
  "steps": [
    { "step": "installTheme", "themeData": { "resource": "wordpress.org/themes", "slug":
"twentytwentyfive" },
      "options": { "activate": true } },
    { "step": "setSiteOptions", "options": { "blogname": "Nyt site" } },
    { "step": "login", "username": "admin", "password": "password" }
  ]
}
```

Start den med `npx @wp-playground/cli@latest server --blueprint=blueprint.json`. Fordi filen ligger i repoet, er en demo ikke længere noget nogen har "liggende på sin maskine" — den er reproducerbar.

Grænsen: Playground er til demo, test og prototyping. Det er ikke drift: intet persistent backend, ingen rigtige betalinger, ingen garanti for at et plugin med server-afhængigheder opfører sig som i produktion.

1.2 Lokal udvikling: vælg ét værktøj og hold fast

Værktøj	Bedst til	Bemærkning
wp-env	standard, CI-venligt	Docker; konfigureres i .wp-env.json
DDEV	teams der vil have mail, DB-værktøjer, HTTPS	Docker; ddev start
WordPress Studio	den der vil have en app og et klik	Automattic-værktøj, mindre CI-orienteret
Playground CLI	hurtig test af plugin/tema	ingen persistens uden mount

Vælg **ét** og skriv det i repoets README. Det vigtige er ikke værktøjet, men at PHP-versionen og

WordPress-versionen i det lokale miljø **matcher drift** — ellers tester man noget andet end det man udgiver.

1.3 Tema: block theme er standarden, og 7.1 gør den stærkere

WordPress 7.1 landede 19/8 2026. For et nyt site betyder det tre ting:

- **Editoren er altid iframed** — også for klassiske temaer og legacy meta boxes. Man skal ikke længere teste "med og uden iframe".
- **Responsive styles i theme.json** under `@mobile` og `@tablet`, og temaet kan selv sætte `settings.viewport` (standard 480 px og 782 px). Mobile CSS er ikke længere et hack i et child theme.
- **Pseudo-states** (`:hover`, `:focus`, `:focus-visible`, `:active`) kan defineres i `theme.json` for Button og Navigation Link.

WooCommerce kræver et **block theme** for at give adgang til hele Site Editor, inklusive produkt- og katalogskabeloner, og kurv/kasse-blokkene er standard for butikker startet efter 8.3. Order storage er **HPOS** som standard for nye installationer siden 8.2 — det skal man ikke slå til, men man skal kontrollere at hvert plugin erklærer sig kompatibelt.

Page builders er ikke forbudt, men de er et valg med en regning: designet ender i databasen, ikke i Git, og hver builder tilføjer sin egen HTML, CSS og JavaScript. Til et site med en fast designidentitet er `theme.json` + patterns både hurtigere og lettere at rulle tilbage.

1.4 Deploy: to ærlige veje

Vej	Sådan	Styrke	Pris
Hostens Git-deploy	git push til et remote, hosten tjekker ud	intet CI at vedligeholde	afhængig af host; nogle låser Test/Live
CI over SSH	GitHub Actions → rsync/SSH → staging	samme pipeline uanset host	kræver nøgler og en pipeline

Konkret, målt 20/10 2026: **WP Engine** har GitPush og en officiel GitHub Action (men `wp-config.php` og `wp-content/uploads` kan ikke pushes), **Pressable** har GitHub-integration, **WordPress.com** har GitHub Deployments på Business og Commerce, **Kinsta** har Git installeret men **ingen push-to-deploy** (man puller over SSH eller kører CI), **Pantheon** kører Dev → Test → Live hvor Test og Live er låste, og **Cloudways** har git auto-deploy via webhooks. Fælles for dem alle: **databasen er ikke i Git.** Indhold, ordrer og indstillinger lever i databasen, og en deploy flytter kun kode.

Derfor er reglen: kode deployes automatisk, **indhold** migreres bevidst. En `staging -> live` -synkronisering af databasen kan overskrive kundeordrer, hvis den køres uden at tænke.

1.5 Bedrock og Composer: kun når man har brug for det

Bedrock (Roots) lægger WordPress i en `web/wp`-mappe, lader Composer styre core og plugins, og bruger `.env` til miljøvariabler. Core og plugins er **ikke** i Git; kun egen kode og `composer.json`/`composer.lock`. Det giver en ren historik og reproducerbare versioner.

Det kræver til gengæld SSH eller en host der accepterer strukturen, og det kræver at man vælger hvem der må opdatere core: Composer eller WordPress' auto-opdatering. **Beslutningen skal træffes før første deploy**, ellers slås de to systemer om de samme filer.

Tommelfingerregel: brug Bedrock hvis I har mere end et par egne plugins eller flere udviklere. Ellers er et almindeligt repo med `wp-content/themes/<tema>` og `wp-content/mu-plugins/` nemmere at drive.

1.6 Drift: det der gør metoden værd at betale for

- **Backup + testet restore.** En backup der aldrig er gendannet, er en formodning. Gendan til staging hvert kvartal og skriv det ned.

- **Staging med noindex og password.** Ellers konkurrerer to kopier af sitet om de samme søgeord.
- **Opdateringer først på staging.** Plugin-opdateringer er den hyppigste årsag til hvide sider.
- **WAF og CDN foran**, men med en kendt måde at måle uden om cachen.
- **Logs og fejlsporing** uden for webroden, så en fatal fejl ikke kun findes i browseren.
- **WP-CLI og SSH**, så en fejl kan diagnosticeres i stedet for at gættes.

---

2. Vores metode, som den faktisk er

Det her er ikke en karikatur; det er den målte tilstand pr. 8/11 2026, og flere af delene er gode.

Faktum	Værdi	Kilde
Host	Simply, delt hosting	docs/agent-profil.md §1
Stack	WordPress 7.1.4, PHP 8.5.12, WooCommerce 11.2.2, 22 aktive plugins	docs/roadmap.md §2
Adgang	wp-admin + MCP (Novamira/AI Engine); ingen SSH/Git i driften	docs/agent-profil.md §4
Kode i Git	plugin-kilde, mu-plugins, CSS, dokumentation — ikke hele sitet	README.md
Deploy	GitHub gør zip'en tilgængelig; WordPress installerer den	plugins/steamgears-autopilot/README.md
Staging	findes ikke	docs/library/deploy-roadmap-2026-11-03.md
Rollback	kladde, revision, fil-backup, option, gammel zip fra git	deploy-playbook §7
Backup	fil- og DB-backup taget 23/10 og 27/10 2026	backup-2026-10-23/, backup-2026-10-27/
Måling	6 projekter med beregnede procenter, 148 historik-rækker	projektstatus.json
Værn	commit-besked, status-drift, fejlscan, PDF-render, smoke-tests	.github/workflows/

Det der er stærkt, og som skal med videre:

- **Dokumentation som hukommelse.** Hver fælde bliver et dokument med dato, bevis og en regel. Det er derfor denne guide kan skrives.
- **Måling før påstand.** Procenter beregnes, ikke gættes; 100 % kræver `verified: true`.
- **Automatiske værn** på commit-form, status-drift, zip-integritet og PDF-render.
- **Agent-adgang over MCP** gør gentagne opgaver scriptbare uden at give en agent SSH.
- **Billig drift.** Et lille site som dette skal ikke koste som en mellemstor virksomhed.

Det der koster, og som den nye metode løser:

Svaghed	Hvad den har kostet
Ingen staging	en plugin- eller indstillingsfejl rammer kunderne direkte
Ingen kode-Git for sitet	en ændring i wp-admin kan ikke diffes eller rulles tilbage
Cache foran PHP	en rettet fatal fejl så ud som en fortsat nedbrudt side
WAF og bot-filtre	målinger kan ramme en challenge i stedet for sitet
Plugin-mappe = ejerens klik	en release er ikke live før et menneske trykker
Ingen lokal reproduktion	fejl findes i produktion, ikke før

---

3. Sammenligning

Område	Vores metode	Git- og staging-metoden	Konsekvens
Hosting	delt hosting, billig	managed WP eller VPS med SSH	pris op, men også værktøjer
Adgang	wp-admin + MCP	SSH, WP-CLI, Git, MCP	fejl findes i logs
Site-kode	i databasen og filer	i Git	diff og review
Deploy	zip/klik	merge eller git push	sekunder vs. minutter
Staging	nej	klon med ét klik	test før live
Rollback	revision/fil/option	forrige release	minutter vs. manuel
Backup	manuel fil/DB	daglig + restore-øvelse	gendannelse er prøvet
Secrets	i options og wp-config	.env/host-secrets	rotation og lækage
Lokal reproduktion	nej	ja	fejl før produktion
Cache	W3TC i WordPress	edge/CDN + object cache	kræver purge-rutine
Sikkerhed	WAF hos host, 22 plugins	managed WAF + færre plugins	mindre angrebsflade
Ændringshastighed	høj for indhold, lav for kode	høj for begge	flere leverancer
Kompleksitet	lav	middel	kræver kompetence
Pris pr. måned	lav	2–10× (estimat)	skal vejes mod timer

Læs rækkerne som en prioritering, ikke en dom. Den største enkeltforbedring er **staging + Git-deploy**; den næststørste er **lokal reproduktion**. Resten er optimering.

4. Hvad »bedre« skal betyde — måltal

Et nyt site skal måles på det samme, uanset metode, så "bedre" ikke bliver en mening:

Måltal	Hvorfor	Baseline at slå
Deploy-frekvens	hvor ofte kan vi levere?	steamgears: content dagligt, kode ved release
Lead time	fra beslutning til live	en side: 3–5 timer (målt)
Change failure rate	hvor ofte brækker en ændring?	historik: 3 hændelser på 14 dage
MTTR	hvor hurtigt er vi tilbage?	1 t 38 min ved butiksnedbruddet 6/11
HTML og scripts	payload	75 KB / 27 scripts (forside)
TTFB og CWV	oplevet hastighed	ikke målt med Lighthouse i dag
Restore-tid	kan vi gendanne?	ikke målt
Fejl i loggen	500'er pr. uge	ikke målt

De to sidste er huller i vores nuværende metode, og de er billige at lukke på et nyt site fra dag et.

5. Beslutningsmatrix — hvad skal det nye site have?

Site-type	Metode	Host	Begrundelse
Portfolio, brochure, 5–15 sider	block theme + Git + staging	managed WP eller delt med Git	ingen kode-tung drift

Site-type	Metode	Host	Begrundelse
Webshop (som steamgears)	Git + staging + object cache	managed WP med SSH	checkout må ikke testes i live
Indholdssite, daglige publiceringer	Git til tema, staging til opdateringer	managed WP	plugin-opdateringer er risikoen
Medlemskab/betaling	Git + staging + testmiljø	managed WP eller VPS	fejl koster penge
App, flere kanaler	headless (Next.js + WPGraphQL)	separat frontend	kun når kanalerne kræver det
Legeprojekt under 500 kr./md.	delt hosting + Playground til demo	delt	accepter: ingen staging

Headless er ikke standarden. Det giver mening når indholdet skal bruges i en app, et andet site eller en kanal WordPress ikke selv kan vise — ellers betaler man for en ekstra frontend uden at få noget igen.

6. Playbook: nyt site i otte faser

Fase	Arbejde	Tid	Gate
0	Formål, målgruppe, indholdsmodel, KPI	2–4 t	én sætning om hvem siden er for
1	Prototype i Playground (Blueprint i repo)	2–6 t	kunden har set den i browseren
2	Host, staging, backup, PHP/WP-version	2–4 t	staging findes og er noindex
3	Repo, lokal opsætning, CI-lint	3–6 t	git push giver grøn lint
4	Block theme, theme.json, patterns, Woo-skabeloner	2–5 d	designet er i Git, ikke i editoren
5	Indhold, metadata, schema, sitemap, redirects	1–3 d	0 døde links, 1 H1 pr. side
6	Staging-test: kasse, mails, formularer, ydelse	0,5–1 d	testordre gennemført
7	Go-live: DNS, 301, sitemap, GSC, uptime	2–4 t	smoke-test efter DNS-skift
8	Drift: uge/måned/kvartal + dokumentation	løbende	restore-øvelse gennemført

Total: 5–9 arbejdsdage for et lille site med eksisterende copy, **2–4 uger** hvis indholdet skal skrives. Det er den samme størrelsesorden som steamgears' deploy-roadmap, men fordelt på et site der ikke findes endnu.

Fase 1 — prototypen er kontrakten

Blueprint-filen er det første commit i repoet. Den giver tre ting: kunden ser sitetet før der er købt hosting, udvikleren får samme udgangspunkt hver gang, og demoen kan genskabes om et år. Læg den i `playground/blueprint.json` og skriv kommandoen i README.

Fase 2 — hosten vælges på seks krav, ikke på pris

1. **SSH + WP-CLI** (ellers kan I ikke fejlfinde).
2. **Git-deploy** — indbygget eller via CI.
3. **Staging** der kan klones og skubbes til live.
4. **Backup** med restore til staging.
5. **PHP 8.5** og en opdateringspolitik I kan leve med.
6. **EU-region** hvis kunderne er i EU (og en databehandleraftale).

Fase 3 — repo og CI

Mindste repo der stadig er professionelt:

```
site/
  wp-content/themes/<tema>/      # block theme, theme.json, patterns
  wp-content/mu-plugins/         # små, uafhængige filer
  wp-content/plugins/<egen>/     # hvis eget plugin
  playground/blueprint.json      # demo og test
  .github/workflows/lint.yml     # php -l, phpcs, JS/CSS-build
  .github/workflows/deploy.yml   # staging, derefter manuel promote
  docs/                          # beslutninger, målinger, fælder
  README.md                      # lokal opsætning, kommandoer, ejerporte
```

Branch-model: `main` = produktion, `staging` = det der er på staging, `feature/*` = arbejde. Releases tagges (`v1.2.0`), så rollback er et tag og ikke en fil man leder efter.

Secrets: kun i hostens secret-store eller GitHub Secrets. Aldrig i `.env` der committes, aldrig i en option der kan læses udefra. En nøgle der har været eksponeret, tilbagekaldes **før** en ny oprettes.

Fase 4 — temaet

- `theme.json` ejer farver, typografi, spacing, `settings.viewport` og responsive varianter.
- **Patterns** til sektioner der gentages (hero, trust, produktgrid), så indhold ikke bliver copy-paste.
- **Skabeloner** til forside, arkiv, produkt, kurv, kasse og 404.
- Ingen hardkodet tekst i temaet; al redigerbar tekst er blokke eller patterns.
- Kontrast og fokusring testes, før designet godkendes — ikke efter.

Fase 5 — indhold og SEO

Samme regler som deploy-playbook-2026-11-08.md §3.2: én H1, meta 120–160 tegn, canonical, ét OG-billede, schema der matcher den synlige tekst, og et sitemap der faktisk indeholder siden. Forskellen er, at det nu kan kontrolleres på **staging** i stedet for efter publicering.

Fase 6 — staging-testen

Test	Krav
Testordre	gennemført, annulleret, noteret
Betaling	mindst én godkendt og én afvist transaktion
Mails	ordrebekræftelse, fortrydelse, retur
Formularer	indsendelse målt, ikke antaget
Consent	afvisning starter ingen ikke-nødvendige scripts
Cache	kurv og kasse er ikke cachede
Performance	budget målt på staging, ikke gættet
noindex	staging blokerer søgemaskiner

Fase 7 — go-live

DNS med lav TTL i god tid, 301 fra gamle URL'er, sitemap indsendt, GSC og analytics verificeret, uptime-overvågning tændt, og en smoke-test **efter** DNS-skiftet — ikke før.

---

7. Startpakke — kommandoer

```
# 1. demo/prototype uden server
npx @wp-playground/cli@latest server --blueprint=playground/blueprint.json

# 2. lokalt miljø (vælg ét)
npx @wordpress/env start                    # kræver Docker + .wp-env.json
# eller: ddev config --project-type=wordpress && ddev start

# 3. hvis I vælger Composer-strukturen
composer create-project roots/bedrock my-site

# 4. tjek kode før push
find wp-content -name '*.php' -exec php -l {} +
```

En CI-deploy til staging er bevidst ikke skrevet færdig her: den afhænger af hosten, og en kopi-klistret `rsync`-kommando uden kendskab til stier og nøgler er farligere end ingen pipeline. Vælg host først, og skriv deploy-jobbet med hostens egen dokumentation ved siden af.

---

8. Hosting: kriterier og shortlist

Der er ingen valgt host endnu. Priser og planer ændrer sig, så de skal hentes hos den enkelte host, når beslutningen tages — ikke citeres fra denne guide.

Host	Git-deploy	SSH	Staging	Bemærkning (målt 20/10 2026)
WP Engine	Ja, GitPush + GitHub Action	Ja, gateway	Ja	wp-config.php og uploads kan ikke pushes
Pressable	Ja, GitHub-integration	Ja	Ja	integrationen er slået til for nye konti
WordPress.com	Ja, Business/Commerce	Ja, visse planer	Ja	GitHub Deployments
Kinsta	Nej, men Git + SSH	Ja	Ja, standard og premium	ingen push-to-deploy; CI eller pull
Raidboxes (DE)	Via SFTP/SSH og CI	Ja	Ja, gratis + add-on	staging deler ikke PHP/WP-version med live
Cloudways	Ja, auto-deploy via webhook	Ja	Ja	mere VPS end managed WP
Pantheon	Ja, Dev → Test → Live	Begrænset	Ja, indbygget	Test og Live er låste
Simply (nuværende)	Nej	Ikke brugt i driften	Nej	billig, kendt, ingen kode-deploy

Sådan vælges: skriv de seks krav fra §6 ned, og stryg enhver host der ikke opfylder dem. Vælg derefter på support, region og pris — ikke på logo.

---

9. Hvis et eksisterende site skal flyttes

Flyt ikke for flytningens skyld. Flyt når der er en konkret gevinst: staging, kode-deploy, ydelse eller sikkerhed. Rækkefølgen er:

1. Frys indhold, tag fil- og DB-backup, og **test gendannelsen**.
2. Opret staging på den nye host, importér, og ret URL'er med `wp search-replace`.
3. Test kasse, mails, formularer, consent og 301'er på staging.
4. Sænk DNS-TTL, skift, og kør smoke-test.

5. Behold den gamle host i 30 dage.
6. Dokumentér hvad der blev ændret, og hvad der stadig er manuelt.

Risici der skal navngives: plugin-licenser bundet til domænet, serialiserede URL'er i databasen, mail-flow der peger på den gamle host, WAF-regler der blokerer den nye, og betalingswebhooks der skal skiftes.

---

10. Fælder i den moderne metode

Den nye metode har sine egne fælder. De står her, så de ikke skal opdages på et nyt site.

Fælde	Hvad der sker	Regel
DB i Git	databasen committes eller overskrives	kode deployes, indhold migreres
Staging indekseret	to kopier konkurrerer i Google	noindex + password fra dag et
Composer vs. auto-update	core og plugins skiftes af to systemer	vælg ejer af opdateringer før deploy
Hostens plugin-forbud	et plugin kan ikke installeres	læs forbudslisten før arkitekturen vælges
Cache skjuler fejl	rettelsen ser ud som en ny fejl	purge før konklusion
Playground som produktion	ingen persistens, ingen betalinger	kun demo og test
theme.json uden review	én ændring flytter hele designet	styles i Git + style revisions
Deploy af uploads	billeder overskrives	uploads er data, ikke kode
Manglende restore-øvelse	backup findes, gendannelse gør ikke	kvartalsvis restore til staging

---

11. Tjekliste før launch

Gruppe	Punkt
Domæne	DNS, TTL, HTTPS, www → apex (eller omvendt), e-mail virker
Adgang	SSH nøgle, WP-CLI, staging-adgang, mindst to administratorer
Backup	daglig backup, off-site, restore testet
Sikkerhed	WAF, 2FA, ingen eksponerede nøgler, mindst mulige plugins
Ydelse	HTML/script-budget målt, object cache, kurv/kasse ikke cachet
SEO	1 H1 pr. side, meta, canonical, sitemap, 301'er, OG-billeder
Butik	testordre, betaling, fragt, moms, mails, returformular
Jura	CVR, adresse, telefon, fortrydelse, consent, GPSR
Måling	GA4 uden dubletter, GSC, uptime, fejl-log
Drift	deploy-rutine, rollback-rutine, kontaktvej, dokumentation

---

12. Hvad vi tager med fra steamgears

Uanset host og metode skal det her følge med, fordi det er det der gjorde steamgears styrbart:

- **Et dokument per fælde**, med dato, symptom, årsag og værn.
- **Beregnet fremgang** — ingen procent uden regnestykke.
- **Maskinelle værn** i CI: form, drift, integritet, tests.
- **Måling før påstand**, også når svaret er "ikke målt".

- **Ejerporte** skrevet ned, så en blokering ikke bliver et gæt.
- **Sproget**: dansk i dokumentation og commits, engelsk på sitet.

13. Kilder

Kilde	Hvad den bruges til	Dato
Playground Handbook	Playground, Blueprints, lokal udvikling	2/11 2026
Playground CLI	npx @wp-playground/cli ... server	4/11 2026
Blueprints: Steps	JSON-skema for login, installTheme, setSiteOptions	14/10 2026
What's new for developers, august 2026	WordPress 7.1, iframed editor, settings.viewport, Site Manager API	8/2026
Theme Handbook: Tools and Setup	lokale miljøer: wp-env, DDEV, Studio	2023
Block themes	hvad et block theme er	—
WooCommerce: Using Blocks	block theme kræves til fuld store editing	2024
WooCommerce HPOS	HPOS er standard fra 8.2	—
Bedrock	Composer-struktur, .env, Git	—
Trellis deployments	zero-downtime deploy og rollback	—
15 hosts checked for Git deploy	hostmatrix: Git, SSH, staging	20/10 2026
Kinsta staging	staging, SSH, Git uden push-to-deploy	15/6 2026
Raidboxes staging	EU-host, staging og begrænsninger	28/7 2026

Ikke undersøgt her: Lighthouse/Core Web Vitals for et nyt site, priser og planer, danske databehandleraftaler, og hvordan WooCommerce 11.3 opfører sig i praksis. Det skal måles, når hosten er valgt.